USGS API Calls Used for the Georgia Seismic Design Table

All numbers in this dataset come from two free USGS web services. One call per point gets the design values; one call per point gets the characteristic earthquake (Modal Mw, Modal R). Substitute `LAT` / `LON` for the point's coordinates, `CLASS` for the site class letter (A–E), and `VS30` for that class's Vs30 (table below). Both use the 2018 national hazard model, PGA, and the 1,033-year (7%-in-75-year) return period, fixed in the calls themselves.

Design values (As, Sd1, PGA, Ss, S1, Sds, design category):

```
https://earthquake.usgs.gov/ws/building-codes/aashto-2009/calculate?latitude=LAT&longitu
↪ de=LON&siteClass=CLASS
```

Characteristic earthquake (Modal Mw, Modal R):

```
https://earthquake.usgs.gov/ws/nshmp/conus-2018.R1/dynamic/disagg/LON/LAT/VS30/1033?imt=
↪ PGA
```

Site Class	`VS30` (m/s)
A	1500
B	1080
C	530
D	260
E	150

Example, Atlanta (33.75, -84.39), Site Class D:

```
https://earthquake.usgs.gov/ws/building-codes/aashto-2009/calculate?latitude=33.75&longi
↪ tude=-84.39&siteClass=D
```

```
https://earthquake.usgs.gov/ws/nshmp/conus-2018.R1/dynamic/disagg/-84.39/33.75/260/1033?
↪ imt=PGA
```

A minimal script

Takes a CSV with `lat` and `lon` columns, calls both services for every row, and writes one output CSV. Set `CLASS` and `VS30` together at the top first (table above). No extra setup beyond Python 3 — only its built-in tools.

```
#!/usr/bin/env python3
"""Usage: python3 fetch_one_class.py input.csv output.csv
input.csv needs "lat" and "lon" columns. Set CLASS and VS30 below first."""
import csv, json, sys, time, urllib.request

CLASS, VS30, RETURN_PERIOD = "D", 260, 1033  # change CLASS + VS30 together

def fetch(url):
    with urllib.request.urlopen(url, timeout=60) as r:
        return json.load(r)

def one_point(lat, lon):
    design = fetch(
        f"https://earthquake.usgs.gov/ws/building-codes/aashto-2009/calculate"
        f"?latitude={lat}&longitude={lon}&siteClass={CLASS}")["response"]["data"]
    disagg = fetch(
        f"https://earthquake.usgs.gov/ws/nshmp/conus-2018.R1/dynamic/disagg/"
        f"{lon}/{lat}/{VS30}/{RETURN_PERIOD}?imt=PGA")
    mode = next(s for s in disagg["response"]["disaggs"][0]["data"][0]["summary"]
                if s["name"] == "Mode (largest m-r bin)")["data"]
    mw = next(d["value"] for d in mode if d["name"] == "m")
    r_km = next(d["value"] for d in mode if d["name"] == "r")
    return {"lat": lat, "lon": lon, "As": design["as"], "Sd1": design["sd1"],
            "PGA": design["pga"], "Ss": design["ss"], "S1": design["s1"],
            "Sds": design["sds"], "Zone": design["sdc"], "Modal_Mw": mw, "Modal_R_km":
             ↪  r_km}

with open(sys.argv[1], newline="") as fin, \
     open(sys.argv[2], "w", newline="") as fout:
    writer = None
    for row in csv.DictReader(fin):
        result = one_point(row["lat"], row["lon"])
        if writer is None:
            writer = csv.DictWriter(fout, fieldnames=result.keys())
            writer.writeheader()
        writer.writerow(result)
        time.sleep(0.5)  # be polite to USGS
```

Run it with: `python3 fetch_one_class.py your_points.csv results.csv`. This version asks both services directly for every point, no shortcuts, so for a large file (thousands of points) it will take a while and may need to be restarted if USGS briefly pushes back.